

1. Автор

Хингелов Глеб Владимирович, группа БПИ252.

2. Условие

Нужно смоделировать морской бой двух игроков. Судья и оба игрока — самостоятельные участники модели. Судья хранит оба поля, принимает выстрелы и сообщает результат. Игрок поля врага не видит, ему приходит только ответ судьбы: мимо, попал или убил.

3. Сценарий работы

1. Программа читает параметры (по умолчанию, из файла `-с`, из командной строки `--ключ=значение`) и проверяет их.
2. Судья расставляет по 10 кораблей каждому игроку (4×1 , 3×2 , 2×3 , 1×4).
3. Игроки по очереди стреляют. Судья применяет выстрел, проверяет своё состояние и печатает ход.
4. Бой кончается, когда все корабли одного игрока убиты, сработал лимит ходов или пришёл сигнал.
5. Печатаются поля с кораблями, победитель и статистика (выстрелы, попадания, убитые корабли).

4. Модули

- `main.c` — параметры, обработка сигналов, коды завершения.
- `judge.c` — судья: цикл боя, вывод на экран и в журнал.
- `board.c` — поле: расстановка, выстрел, проверка правил.
- `player.c` — игроки: стратегии `random`, `hunt` и `human`.
- `game.h` — общие типы.

Всего около 540 строк кода и ещё около 190 строк тестов. Ввод и вывод через дескрипторы (`open`, `read`, `dprintf`), без `printf` и `fopen`.

5. Параметры

`size` (8–16), `touch` (1 — корабли могут касаться), `again` (1 — после попадания ход ещё раз), `max_moves`, `delay` (мс), `seed`, `a` и `b` (стратегии `random`, `hunt` или `human`), `log` (файл журнала, `none` — не писать). Готовые файлы настроек: `configs/classic.cfg`, `configs/touch_switch.cfg`.

Параметр `games` — сколько боёв сыграть подряд (в конце печатается сводка: победы, среднее, минимум и максимум ходов; первым ходит по очереди то один игрок, то другой). Стратегия `human` — ход вводит человек с клавиатуры (B5, q — выход), ввод читается через `read(0)`.

6. Запуск и завершение

```
mkdir -p build && cd build && cmake .. && cmake --build . && ctest
./bin/battleship
./bin/battleship -c configs/classic.cfg --seed=5
```

Коды завершения: 0 — бой закончен, 1 — неправильные параметры, 2 — не открылся файл настроек, 3 — не расставился флот, 4 — судья нашёл нарушение правил, 130 — Ctrl+C (SIGINT), 143 — SIGTERM. При сигнале бой останавливается, печатается итог и закрывается журнал.

7. Результаты

Пример вывода:

Ход 110. Нельсон(hunt) -> G8 УБИЛ

...

Победил Нельсон(hunt) за 110 ходов

Нельсон(hunt) выстрелов 60, попаданий 20 (33%), убито кораблей 10

Ушаков(random) выстрелов 50, попаданий 10 (20%), убито кораблей 2

Тот же бой пишется в battleship.log. С одним seed бой повторяется полностью.

Стратегия hunt в этом примере выиграла у random, что и ожидалось: она добивает раненый корабль, а random стреляет вслепую.

Серия из 1000 боёв (--games=1000, поле 10×10, touch=0): hunt против random выиграл 978 боёв, проиграл 22, в среднем бой длится 105 ходов. Для проверки справедливости сыграны random против random: 482 против 518 побед, в среднем 137 ходов, то есть без перекоса в пользу первого ходящего.

8. Тесты

tests/run_tests.sh (36 проверок, запускается через ctest): бои с разными seed, все сочетания touch и again, оба файла настроек, лимит ходов, повтор боя по seed, плохие параметры (коды 1 и 2), запись журнала, ввод человека, серия боёв, Ctrl+C (код 130). Все проходят.

Модульные тесты на C (tests/unit_tests.c, 1692 проверки): расстановка, выстрелы, игроки random и hunt, поведение hunt после попадания, пометка воды вокруг убитого корабля, границы генератора случайных чисел. Дополнительно программа прогонялась с -fsanitize=address, undefined: ошибок нет.

9. Использование ИИ

ИИ использовался в небольших количествах при составлении отчета, а также при создании некоторых тестов.